Robotnik

# Developer manual

ROS 2 - Jazzy - v2.2

# Contents

# 1. Introduction

Welcome to the Developer manual. This document provides detailed information on the robot parameters, operational procedures and software development using ROS 2.

It is intended for advanced robot programmers with an intermediate level of programming expertise and are well-acquainted with ROS 2.

Prior to operating the robot, it is mandatory to review all safety warnings included in the User manual of the robot, to ensure proper handling and operation.

For additional information, please refer to:

- User manual of the robot
- Maintenance manual of the robot
- Control interface manual of the robot
- Appendices

## 1.1. Contact

Robotnik headquarters:
SPAIN
Robotnik Automation S.L
Ronda Auguste y Louis Lumière, 8
46980 P. Tecnológico, Paterna
Valencia (España)

Should you encounter any issues or have questions regarding your robot, please reach out to us via email at support@robotnik.es. Remember to include your robot's serial number in your correspondence. We are committed to continually improving and upgrading our products and services and we welcome any feedback you may have.

**NOTE:** You may contact us in either English or Spanish.

## 1.2. Robot Types

This manual applies to the following robot models that work with ROS2 Jazzy:

- RB-THERON
- RB-SUMMIT
- RB-STEEL
- RB-KAIROS
- RB-VOGUI
- RB-ROBOUT

NOTE: This manual may include some images from different robots, as visual support of the applicable written information.

# 2. Software architecture

The Robotnik software is based on ROS2 and it is designed with a modular structure that separates the core functionality into three main components. This architecture ensures scalability, maintainability and clarity in system configuration and deployment:

- Launch System
- Robot Software
- Robot Settings

For detailed ROS2 systems descriptions, tutorials and a really important number of stacks and packages, please visit www.ros.org.

All open source software developed by Robotnik Automation can be found by visiting the official Github repository (https://github.com/RobotnikAutomation).

## 2.1. Launch System

The launch system is responsible for loading the Robotnik Software.

It is launched automatically at system startup via a systemd service named *ros-bringup.service*, located in *~/.config/systemd/user/*.

This service runs the script *bringup.sh* located in */home/robot* is responsible for orchestrating the launch of the **Robot Software**.

The *bringup.sh* script also executes *ros_config.sh* located in /home/robot to load the **Robot Settings**. These settings configure and adapt the software to the specific robot model and its components, such as sensors, actuators, and network parameters.

The software is deployed across multiple screens, each corresponding to a virtual terminal that represents a specific module of the system architecture. Within each screen, a ROS 2 launch file is executed to orchestrate the nodes associated with that module.

This process is illustrated in the software startup sequence diagram as follows:

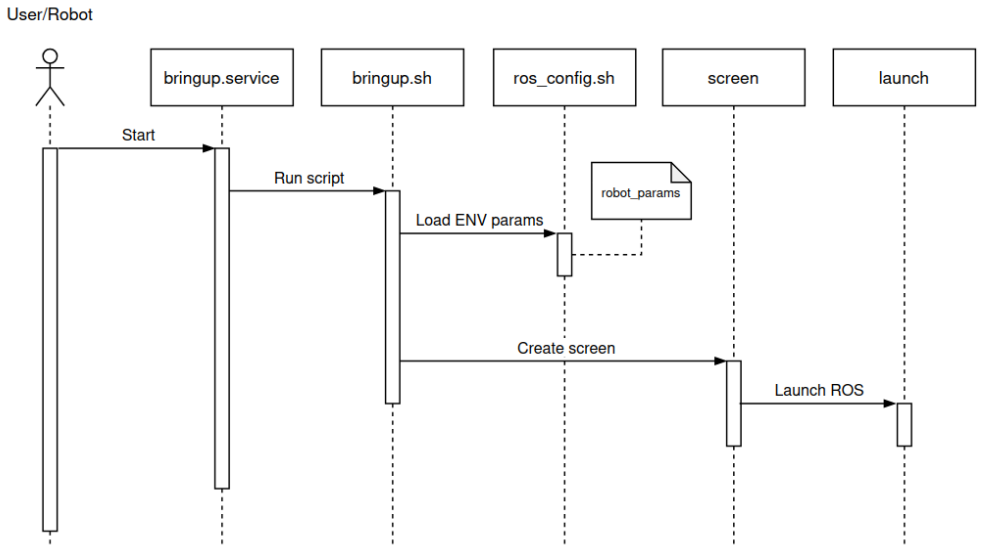


*Figure - Software startup sequence diagram*

| Name | Function | Description |
|---|---|---|
| ros-bringup.service | Starts the Launch System | Initiates Robotnik Software by running bringup.sh. Located at *~/.config/systemd/user/*. |
| bringup.sh | Load Robotnik Software | Runs ros_config.sh and orchestrates the launching of the components of the Robotnik Software. Located at */home/robot* |
| ros_config.sh | Load Robot Settings | Load robot_settings to configure the components launched by bringup.sh. Located at */home/robot* |
| robot_settings | Contains software settings | Configures the software for the robot. Located at */home/robot/robot_settings* |

## 2.2. Robot Software

The core of Robotnik's Software Architecture is **robot_packages**, which contains the main software required to operate the robot. Built on ROS 2, it is structured into multiple modules, each designed to address a specific aspect of the robot's functionality.

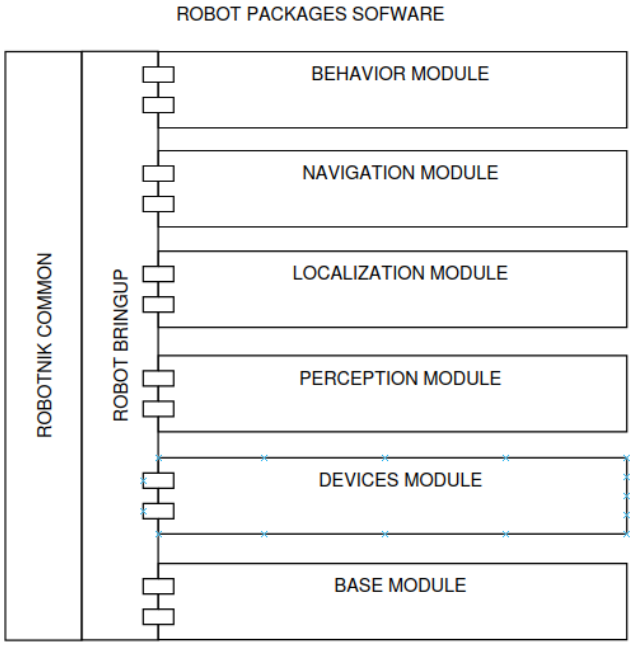


*Figure - Robot Software Architecture*

The *robot_packages* is located in this path:

```
cd /home/robot/robot_ws/src
```

It uses the *robot_bringup* ROS2 package to orchestrate the nodes of the modules through launch files depending on robot_settings configuration. It is located on this path:

```
cd /home/robot/robot_ws/src/robot_packages/packages/robot_bringup
```

The modules define the structure of the **robot_packages** and **robot_bringup** folders. Each module serves a specific purpose:

- **Base**: Ensures the robot's basic motion capabilities and core functionalities common to all robots, such as battery management, IMU orientation and pad control.
- **Devices**: Provides access to the robot's optional accessories, including sensors and devices.
- **Perception**: Processes data from the physical world, translating it into information that other modules can use for decision-making.
- **Localization**: Builds a map of the environment and estimates the robot's position and orientation in the real world.
- **Navigation**: Calculates trajectories for autonomous movement and executes atomic actions such as docking or moving to a specific location.
- **Manipulation**: Provides control for arms, grippers and related components for handling objects.
- **Behavior**: Orchestrates complex pre-defined actions that involve interactions between multiple modules. Examples include charging, unpicking or moving to a specific location.

All modules share common Robotnik packages that are used across different parts of the software. The main packages are:

- **robotnik_common**: A collection of utilities for managing ROS 2 launch files and filesystem structures.
- **robotnik_interfaces**: Custom ROS 2 messages, services and actions defined by Robotnik to standardize communication between modules.

Each module is designed to provide a standardized input and output interface, ensuring consistent communication with other parts of the software.

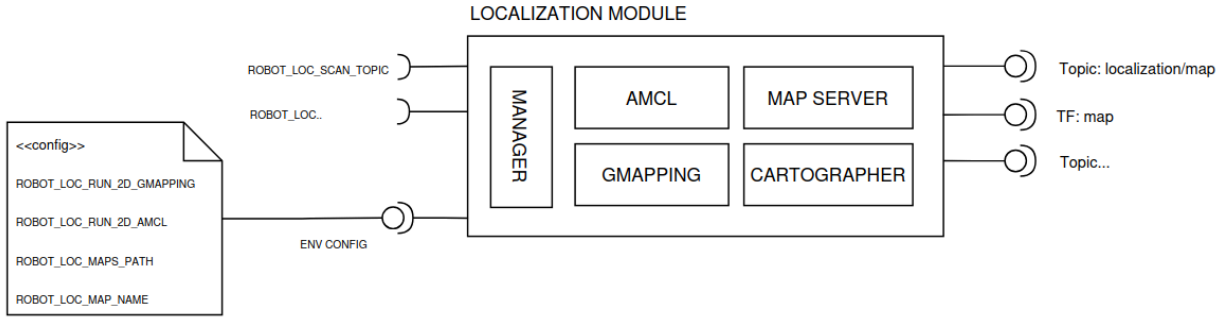


*Figure - Module diagram*

It uses the information of robot_settings to define its interfaces and behavior. There are three types of parameters:

- **Run**: Specifies which software components within the module should be launched.
- **Interfaces**: Defines the input and output topics, services, actions and transforms (TF) required or provided for interaction with other modules.
- **Configuration**: Adjusts the module's behavior during runtime, allowing for flexibility and customization.

Each module is launched inside a virtual terminal called screen. To see the available screens:

```
screen -r
```

Check the log of specific modules by accessing inside of the screen:

```
screen -r base
```

## 2.3. Robot Settings

The robot settings define how the software is launched and set specific configurations such as maps and other persistent data for the user. It is located at ~/robot_settings folder and is divided into:

- **robot_params:** The *robot_params* folder provides the configuration for all robots, organized into different parameter files:

- **Bringup**: Defines general parameters for launching modules, such as common configurations, package references and file paths.
- **Base**: Contains parameters specific to the robot's base module, organized by robot model (e.g., *rbtheron*, *summit_xl*, etc.)
- **Common**: Includes parameters for each module, such as *devices*, *perception*, *localization*, *navigation* and *behavior*.

This structured approach allows for modularity and scalability when configuring different robot platforms. For more information, check the *robot_params* folder.

- **robot_data:** contains persistent data for the user such as maps of file configurations.

# 3. Software development

## 3.1. File and Folder Structure

Robotnik's software is organized using a consistent file and folder structure across all **standard** robots.

- **Host and hostname** are the serial number of the robot.
- The **user directory** is */home/robot*.
- Configuration and launching files are located in the user directory.
- Robot software is located in the user directory in a **workspace** folder called *robot_ws*.
- The workspace folder contains *robot_packages* and might contain *<serial>_description* and *<serial>_moveit* packages.
- The robot's URDF might be located either in the *robot_settings* folder or in *robotnik_description*, under the name *<serial>.urdf.xacro*.
- The robot_settings is located in the user directory.
- A backup of the workspace, robot_settings, UDEV rules folders, etc, is located in the user directory.

This is the typical structure for a standard robot located in the **user directory:**

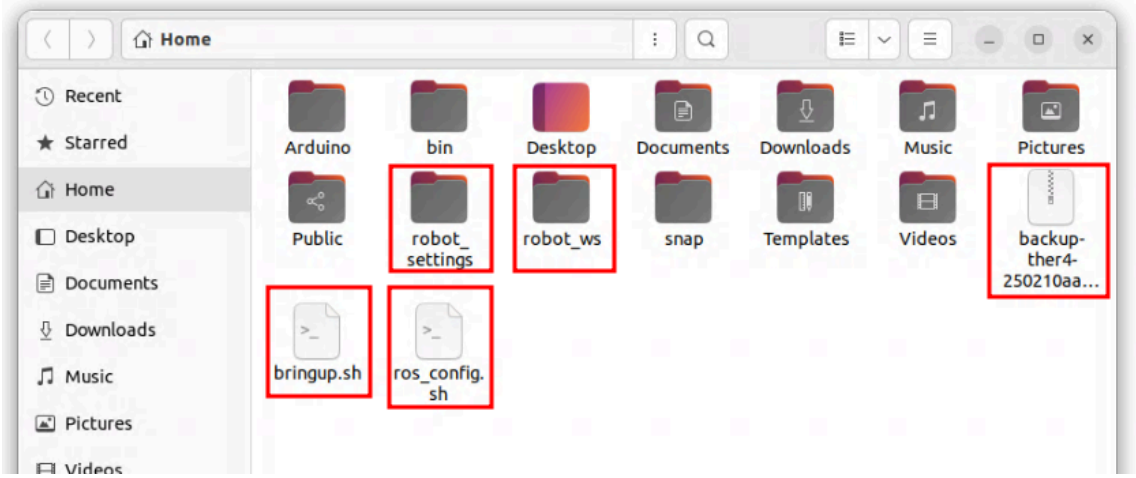


*Figure - Home folder structure*

The **robot software** is inside the workspace folder. It typically has the following structure:

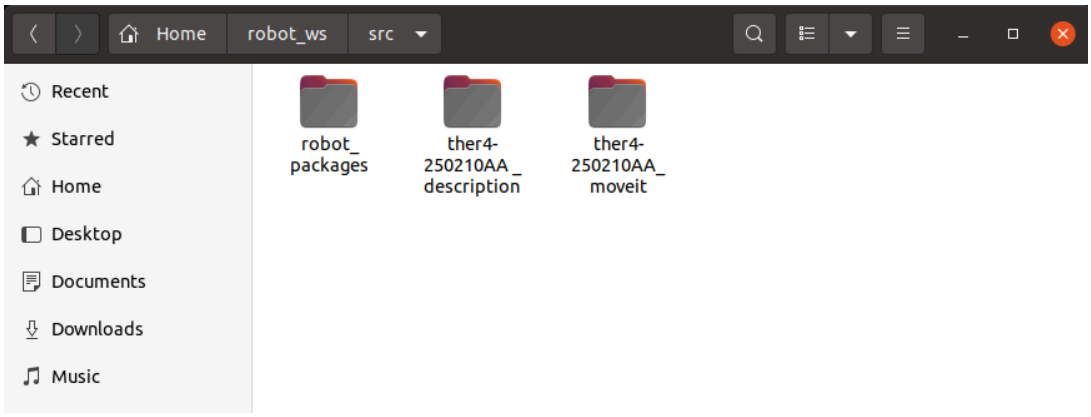


*Figure - robot_ws structure*

Inside the **robot_packages** folder are located the software packages (packages), internal documentation and test scripts (docs and scripts) and deb folder.

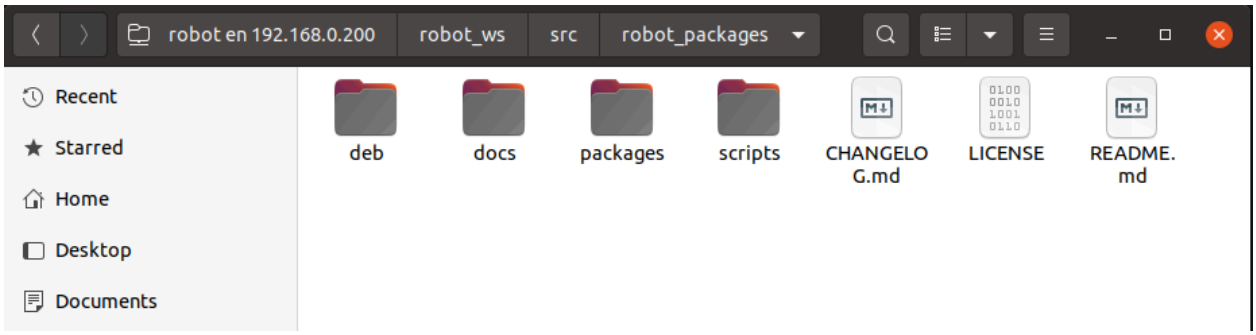


*Figure - robot_packages structure*

Inside the **packages** folder are located the modules (navigation or perception) and common software for all modules (robot_bringup or robotnik_common).

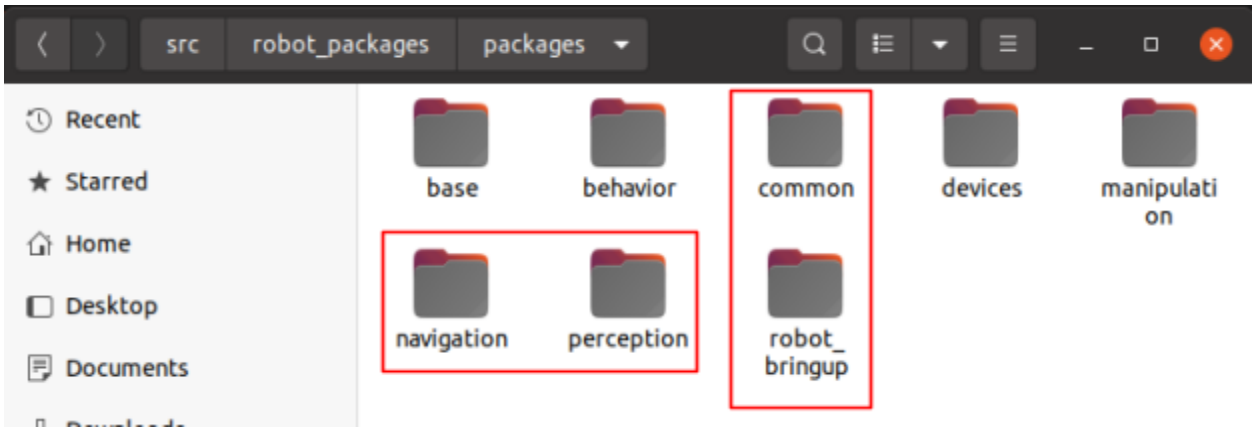


*Figure - packages structure*

## 3.2. Start and Stop Software

To start Robotnik's software, connect to the robot and run the bringup script:

```
~/bringup.sh
```

To stop it, close all screens:

```
killall screen
```

In order to stop a specific module:

```
screen -r <module-name>
Ctrl + C
```

Or:

```
screen -S <module-name> -X stuff $'\003'
```

To restart the software, run again the bringup script:

```
~/bringup.sh
```

## 3.3. Add Custom Software

There are two ways to add new software features:

- **Using an external machine:** This is the **recommended and safest** approach, as it does **not modify** the Linux system where Robotnik's software is deployed. It also **facilitates support and updates**, since the original system is preserved as delivered from the factory.

- **Inside the robot CPU:** If you need to develop directly on the robot's CPU, it is recommended to **use Docker** for deploying new features. If you choose to work directly on the **host system**, please be cautious **not to compromise the system**. A backup of Robotnik's software is provided in the user directory.

# 4. System Access

To connect and configure the robot, there are multiple ways to address it.

## 4.1. Network Connection

Refer to the User Manual for instructions on how to connect to the robot.

## 4.2. Access Methods

There are three methods to access the robot:

- **SSH**: for terminal access
- **RDP**: for remote desktop access
- **ROS 2 DDS**: for communication via topics, services and actions

SSH and RDP access are covered in the User Manual. This chapter focuses on the ROS2 connection.

### 4.2.1. ROS2 DDS Communication

Access to ROS2 topics, services and actions through DDS from **an external machine.**

1. Install ROS 2 Jazzy Jalisco on Ubuntu 24.04 machine. (https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html)

2. Install dev tools and ROS middleware:

```
sudo apt-get install ros-dev-tools
sudo apt-get install ros-jazzy-rmw-cyclonedds-cpp
```

3. Create a workspace with Robotnik messages and compile it.

```
mkdir ~/myws && mkdir ~/myws/src && cd ~/myws/src
git clone --branch jazzy-devel
https://github.com/RobotnikAutomation/robotnik_interfaces.git
cd ~/myws
colcon build --merge-install --symlink-install
source install/local_setup.bash
```

4. Set ROS_DOMAIN_ID and RMW_IMPLEMENTATION to establish connection:

```
export ROS_DOMAIN_ID=38
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```

5. At this stage, the ROS 2 interface is available for communicating with the robot. For example, to list all topics published by the robot, use the following command:

```
ros2 topic list
```

## 4.3. Monitoring

### 4.3.1. Screen

Screen is a virtual terminal used to run the Robotnik Software modules. There is one screen per software module and the logs can be accessed from the terminal.

To see all available screens:

```
screen -r
```

To check logs from a specific screen:

```
screen -r base
```

To exit the screen, press the following keys <CTRL + A + D>

```
Ctrl + A + D
```

To kill a software module, open a screen and then press:

```
Ctrl + C
```

If the process inside screen dies or is stopped, you can resume it:

```
r
```

### 4.3.2. RVIZ2

RViz2 is a 3D visualization tool for ROS 2. It allows you to visualize the robot model, sensor data and other useful information in a 3D environment.

There are two ways to use RViz2:
- On the robot computer.
- On your PC with the robot connected.

To use RViz2 on the robot computer, you need to be connected to the robot via RDP. To use RViz2 on your local PC with the robot connected, you must follow the instructions described in the ROS 2 Connection chapter.

Then you must be able to execute the following command to launch RViz2:

```
rviz2
```

### 4.3.3. RQT

RQT is a software tool that allows you to monitor and control the robot. It provides a graphical interface, QT-based, that allows the visualization of cameras, graphs and other information.

There are two ways to use RQT:
- On the robot computer.
- On your PC with the robot connected.

To use RQT on the robot computer, you need to be connected to the robot via RDP. To use RQT on your local PC with the robot connected, you must follow the instructions described in the ROS 2 Connection chapter.

Then you can use the following command to launch RQT

```
rqt
```

## 4.4. Robot operation with gamepad

The robot can be operated using the supplied gamepad. The procedure for turning the controller on is described in the main robot user manual. By default, this controller should already be paired with the robot.

The re-pairing procedure should only be used in these cases:

- The controller **no longer connects** to the robot.
- The controller **has been replaced** by a new one.
- The controller was paired with another device and you want to **restore the connection** with its original robot.

### 4.4.1. Repairing the remote controller

Use this procedure when you need to pair a controller with the robot again (for example, after replacing it or after it was paired with another device).

1. **Prepare the controller**
    a. Make sure you have the controller you want to pair physically available.
    b. Ensure the controller is **powered off** before starting the process.

2. **Trigger the pairing process on the robot.**
    You must first tell the robot to start searching for a controller. This can be done in one of two ways:

    a. **Option A: Using USB**.
    Use this option if you have physical access to the robot's USB ports.

    - Connect the controller to the main PC of the robot using a USB cable.
    - Wait a few seconds.
    - Disconnect the controller from the USB port.

b. **Option B: Using SSH**.
Use this option if you have access to a terminal and can connect via SSH.

- Open a terminal on your computer.
- Connect to the robot via SSH:

```
ssh robot@192.168.0.200
```

- Once connected, run:

```
repair-gamepads
```

This command also starts the controller search and pairing process. Example of the output from previous command:

```
$ repair-gamepads
looking for paired gamepads (DualShock, DualSense)
...
```

If want to see more detailed info, see section Troubleshooting

3. **Put the controller into pairing mode**
After triggering the process on the robot (via USB or SSH), put the controller into Bluetooth pairing mode using the appropriate button combination for your model until the controller starts blinking quickly.

- **PS4 controller (DualShock 4):** Press and hold `Share + PS` simultaneously.
- **PS5 controller (DualSense):** Press and hold `Create + PS` simultaneously.

4. **Pairing result**
   - When pairing is successful, the controller LED will **stop flashing rapidly** and remain on with a steady color.
   - Once paired, the controller will automatically reconnect to this robot on the current power cycle and on subsequent startups, as long as it is in range and powered on.
   - If the pairing **process fails**, the controller **will remain off**. In that case, repeat the pairing procedure from the beginning.

### 4.4.2. Important notes

- You must use the **same type of controller** that was used previously (e.g., if you previously used a PS4 controller, you must pair another PS4 controller).

- The system cannot choose which specific controller to pair. If there is any compatible controller in pairing mode nearby, it may be paired instead of the one you intend.

- Starting the repair process removes any previously paired controller, ensuring that only one controller is connected at a time.

- If you want to use a previously paired controller again, you must repeat the pairing process described above.

### 4.4.3. Troubleshooting

In order to debug the process, to verify that is running correctly, you can monitor its log output over SSH:

1. Open a terminal on your computer.

2. Connect to the robot via SSH, for example:

```
ssh robot@192.168.0.200
```

3. Once connected to the robot, run:

```
journalctl -ft bluetooth-gamepad
```

This command shows the real-time log output of the bluetooth-gamepad process which is in charge to perform the pairing automatically.

# 5. Base module

The base module ensures core functionalities common to all robots. It is composed of four parts:

- Motion: description and controllers.
- IMU: imu data orientation.
- Battery management: charge/uncharge management and battery data.
- Pad control: provides manual control of the robot.
- Safety: Connection with a safety PLC.

## 5.1. Motion

The base control is based on the ros2_control and ros2_controllers packages, which divide the developed software between controllers and hardware interfaces. These interfaces are connected by status interfaces and command interfaces, allowing for a main loop and synchronizing all components of the robot.

The controller_manager node loads all needed plugins. There are the following components (note the name is not the same as the loaded plugin):

- **robotnik_base_hw**. Hardware interface to command wheels.
- **robotnik_base_control**. This controller computes kinematics for wheels, applies limits, etc.
- **robotnik_io_control**. This Controller manages the low-level IO module exposed by robotnik_base_hw and drivers.

### 5.1.1. Robotnik Base HW

This plugin is loaded by the controller_manager and is responsible for sending commands to the motors. It is based on the ros2_control and is a subclass of the hardware_interface::SystemInterface class.

The following parameters are described inside URDF description, just containing the hw interface:

| Parameter | Description | Default value |
|---|---|---|
| **device** | SocketCAN interface connected to the motor wheels. | |
| **sync_hz** | Frequency of synchronization of the bus sync in Hz. | |

Joint parameters:

| Parameter | Description | Default value |
|---|---|---|
| **joint_name** | Joint name of the wheel. | |
| **id** | CAN ID of the motor. | |
| **encoder_resolution** | Encoder resolution of the motor. | |
| **gear_ratio** | Mechanic gearbox ratio applied to the wheel | |

Additionally, the hardware interface exposes the following interfaces to command it and read actual values.

| Interface | Description | Units |
|---|---|---|
| **position** | Position of the joint. | rad |
| **velocity** | Velocity of the joint. | rad/s |
| **effort** | Effort of the joint. (kp = 1Nm/A) | Nm |

Avoid any modification to the configuration, as unauthorized changes may result in loss of warranty in case of physical damage, in addition to causing the robot to operate improperly.

### 5.1.2. Robotnik Base Control

This plugin is loaded by the controller_manager and is responsible for computing the kinematics for the wheels, applying limits, and sending the commands to the robotnik_base_hw. It is based on the ros2_control and is a subclass of the controller_interface::ControllerInterface class.

This controller works either with the cmd_vel input or the joint_state input.

General parameters:

| Parameter | Description | Default value |
|---|---|---|
| **profile** | Profile used | base |

The profile is a set of different parameters for the limits in cartesian and wheel velocities and accelerations. By default, there is just one profile called base, which contains a set of parameters called linear and angular (limits for cartesian) and each wheel name (limits for wheels).

Timeout parameters:

| Parameter | Description | Default value |
|---|---|---|
| **cmd_vel_timeout** | If the time from the last msg is larger than this timeout, the robot will stop. | |
| **joint_command_timeout** | If the time from the last msg is larger than this timeout, the robot will stop. | |
| **imu_timeout** | If the time from the last msg is larger than this timeout, the robot will stop using imu data. | |

Topics parameters:

| Parameter | Description | Default value |
|---|---|---|
| **cmd_vel_topic** | Input topic name for cmd_vel | |
| **odom_topic** | Output topic name for odom | |
| **imu_topic** | Input topic name for imu | |
| **emergency_topic** | Input the topic name for the emergency | |
| **joint_control_topic** | Input the topic name for joint control | |

The *robotnik_base_control* node is in charge of receiving raw command velocities, computing the kinematics controller and then commanding the wheels from the *robotnik_base_hw* component.

```
/robot/robotnik_base_control/cmd_vel_unstamped
```

To manage robot control commands safely and avoid concurrent access issues, it's advisable to use the *twist_mux* node's topics instead of directly controlling the robot via the aforementioned topic. The *twist_mux* node acts as a multiplexer for Twist messages. It receives Twist commands from various topics and outputs one of them based on a configurable priority and timeout mechanism.

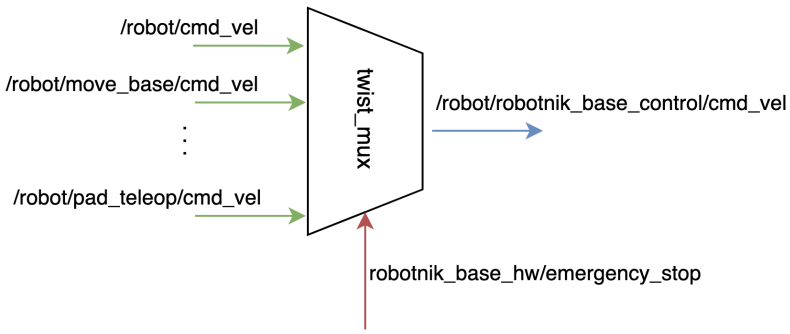


*Figure - twist_mux diagram*

These are the most commonly used topic velocity commands. They are ordered from highest to lowest priority. These are the most common velocity commands for topics, listed from highest to lowest priority.

```
Priority   Input topic
100      -> /robot/pad_teleop/cmd_vel
 30      -> /robot/dock/cmd_vel
 30      -> /robot/move/cmd_vel
 20      -> /robot/move_base/cmd_vel
```

Priority can be configured in *robot_bringup/config/base/twist_mux/twist_mux_topics.yaml*

Each topic is used for a specific purpose:

- **pad_teleop**: usually used by the pad. It has the highest priority. The user can interrupt any other velocity command by pressing the dead man button and sending zero velocity.
- **dock**: topic published by the robotnik_dock package, used by some complex actions such as the autonomous charge and uncharge of the robot.
- **move**: topic published by the robotnik_move package, used by some complex actions such as the autonomous charge and uncharge of the robot.
- **move_base**: topic used by the navigation algorithm.

### 5.1.3. Robotnik IO Control

The plugin is loaded by the controller_manager and is responsible for managing the IO module exposed by robotnik_base_hw. It is based on the ros2_control and is a subclass of the controller_interface::ControllerInterface class.

This plugin exposes ~/io topic, which returns feedback from IO wheels and exposes ~/set_digital_output service to change outputs for your application.

## 5.2. IMU

The Inertial Measurement Unit (IMU) is integrated into the base module, as it is intended to be included across platforms for orientation and odometry estimation. The provided data is required for the *robotnik_base_control* controller

Use the following topic to access the IMU data. This topic provides filtered orientation data in quaternion format.

```
ros2 topic echo /robot/imu/data
```

## 5.3. Battery Management

The battery management system provides battery data such as the battery level and cell voltage. It also controls the relay to enable or disable the battery contactors for charging at the charging station.

Use the following  topic to access the battery data:

```
ros2 topic echo /robot/battery_estimator/data
```

- **Voltage**: The total voltage of the battery pack.
- **Current**: The total current drawn from or supplied to the battery pack.
  **Level**: The overall charge level of the battery.
- **Cell voltages**: The individual voltage measurement for each cell in the battery pack.

Use the following to consult battery contactors:

```
ros2 topic echo /robot/charge_manager/status
```

- **operation_mode**: Indicates how the battery contactor relay is managed. It can be controlled automatically by software (automatic_sw), by hardware (automatic_hw) or manually via a service call (manual_sw).

- **contact_relay_status**: Indicates whether the contactor relay, which enables charging, is open or closed.

- **charger_relay_status**: Indicates whether the robot is correctly positioned at the charging station. In some robots, this status is determined by detecting the magnetic field generated by the charging station near the robot base. **In other ones, this signal is not used**.

## 5.4. Pad Control

Pad control is included as part of the base module to provide a simple manual teleoperation. See the User Manual for more pad control information.

Use the following topic to check the velocity commands published by the joystick/pad. This topic is connected to the *twist_mux* node, which forwards the velocity commands to the motor controllers.

```
ros2 topic echo /robot/pad_teleop/cmd_vel
```

In case of a problem, check the joy topic, which provides the raw data from the joystick/pad.

```
ros2 topic echo /robot/joy
```

## 5.5. Safety module

The safety module is a software component that communicates with the safety PLC and allows sending and receiving information and commands from the hardware safety system. Key features:

⚠ This component is only used **when the robot is equipped with a safety PLC**. Robots without a safety PLC will not have available the topics and services listed below.

⚠ The modification of Robotnik software installation parameters could invalidate the robot safety and the risk assessment, and could affect the guarantee of the robot.

The safety module lets the robot interact with the safety PLC through modbus communication. Some information provided by the PLC is shown in the safety module topics:

- **status** (robotnik_safety_msgs/msg/SafetyModeStatus)

```
ros2 topic echo /robot/safety_module/status
```

Example output:

```
operation_mode: auto
safety_mode: safe
emergency_stop: true
safety_stop: true
current_speed: 0.0
laser_mode: standard
laser_status:
- name: front_laser
  detecting_obstacles: true
  contaminated: false
  free_warning: false
  warning_zones: []
- name: rear_laser
  detecting_obstacles: false
  contaminated: false
  free_warning: false
  warning_zones: []
```

Field meanings:

- **operation_mode**: (*auto*, *manual*, *maintenance*)
- **safety_mode**: (*safe*, *unsafe*)
  In *unsafe*, automatic operation must be avoided by the user.
- **emergency_stop**:
  *true* if the safety system is latched in an emergency stop. Requires user action to reset (check E-stop buttons and press the reset button). *false* if ok.
- **safety_stop**:
  *true* if a safety stop is active but will clear automatically when the hazard is gone. No user action needed.
- **current_speed**:
  Robot speed in m/s fed back to the safety PLC. Computed from odometry.
- **laser_mode**: (*standard*, *charging_station*)
  Current safety laser mode applied.
- **laser_status.detecting_obstacles**:
  *true* if something is inside the active safety field. The field scales with speed. See the user manual.
- **laser_status.contaminated**:
  *true* if the sensor needs inspection or cleaning. See the user manual.
- **laser_status.free_warning**:
  *true* if the warning area is clear. See the user manual.
- **laser_status.warning_zones**:
  List of active warning zones, empty if none.

- **emergency_stop** (std_msgs/Bool): Shows if the emergency stop is active.

```
ros2 topic echo /robot/safety_module/emergency_stop
```

The same way that the safety module lets read information from the PLC, it also allows writing commands to the PLC:

- **set_laser_mode** (robotnik_common_msgs/srv/SetString). Set safety laser modes. The ranges and modes are detailed in the user manual.

```
ros2 service call /robot/safety_module/set_laser_mode
robotnik_common_msgs/srv/SetString "data: 'standard'"
```

These modes are mainly used to reduce the safety area to allow approaching the charging station.

- Watchdog. The main PC software sends a heartbeat signal. If it does not, due to a software hang, the safety PLC must stop and prevent restart safety related.

- Feed back the current speed to the PLC. This happens in the background and the user does not need to worry about it.

- Low-level I/O:

  Read signals:

```
ros2 topic echo /robot/safety_modbus/io -f | grep -i -A1 emergency
```

  Set a digital output:

```
ros2 service call /robot/safety_modbus/set_digital_output
robotnik_io_msgs/srv/SetDigitalOutput "output:
  id: 0
  name: ''
  value: false"
```

Rules:
- *Id* or *name* identifies the signal. *value* is the payload.
- IDs start at 1.
- If id == 0, the service uses name as the selector
- Do not set both id (non-zero) and name at the same time.

# 6. Perception module

This module is responsible for data processing, providing useful information for other modules, such as detection and filtering.

It offers the following services:

- **Locator**: Provides pose estimation of AR markers.
- **Pose Filter**: Filters noise from TF-based position data.

## 6.1. Maker Detection

This algorithm uses OpenCV and the ar_track_alvar library to detect AR markers used mainly for charging stations.

1. In the remote desktop, open RViz2.

2. Place the robot in front of an AR marker, such as the one located in front of the charging station. The marker will create a TF from *robot_odom* as *robotnik_marker_1*. The behavior module uses this TF to identify where the charging station is.

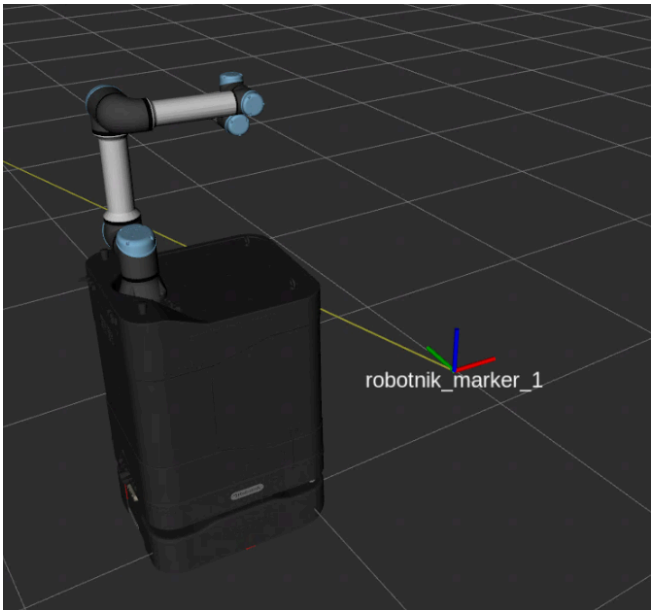


*Figure - Detected marker in rviz2*

3. The detected AR markers can be accessed through the following topic. This topic provides information about their TF frame names and positions.

```
ros2 topic echo /robot/robotnik_locator/object_list
```

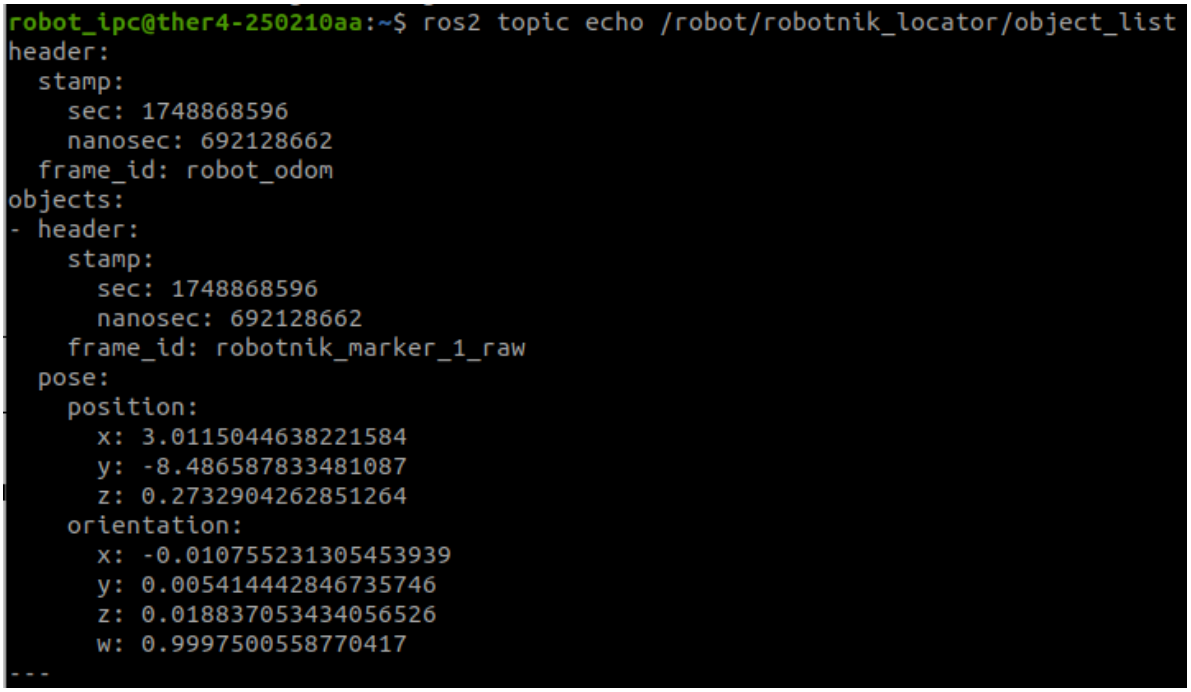


*Figure - Object list output*

# 7. Localization module

This module is responsible for mapping and localization by providing an estimated pose for other modules, such as navigation.

3 different services are running on this module:

- **map_server:** This node will publish the map for localization and navigation.
- **gmapping:** The gmapping package provides laser-based SLAM (Simultaneous Localization and Mapping) based on OpenSlam's Gmapping.
- **amcl:** The amcl package is a laser-based probabilistic localization system. It implements the adaptive (or KLD-sampling) Monte Carlo localization approach, which uses a particle filter to track the pose of a robot against a known map.

**Note:** The following chapters detail how to create and load a map **manually**. Future software releases will include a system to facilitate this process.

## 7.1. Create a Map

### 7.1.1. 2D Map

1. Establish an SSH connection to the robot or open a terminal using a remote desktop via RPD.

2. Activate 2d mapping by setting `ROBOT_LOCALIZATION_MODE` in *robot_params*.

```
sed -i \
  -e 's/^export ROBOT_LOCALIZATION_MODE=.*/export
ROBOT_LOCALIZATION_MODE="mapping_2d"/' \
  "$HOME/robot_settings/robot_params/common/localization_params.env"
```

3. Relaunch Robotnik's software to apply the changes:

```
~/bringup.sh
```

4. In the remote desktop, open RViz2 to verify that the robot is mapping.

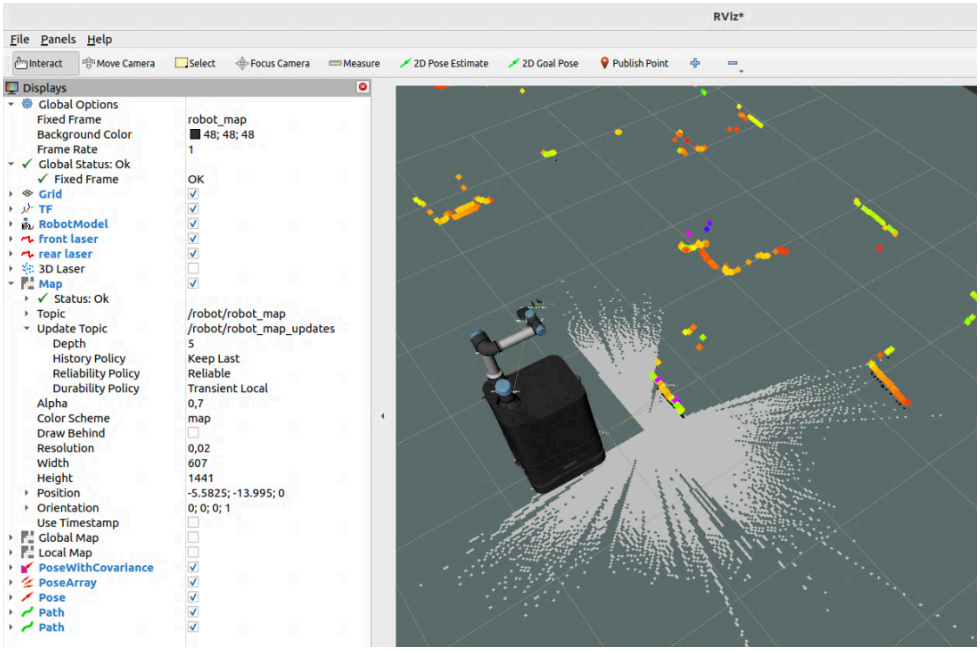


*Figure - Mapping in rviz2*

5. Save the map setting the map name. **Select a unique map name**, otherwise, it will overwrite the existing map.

```
ros2 launch robot_bringup map_saver.launch.py map_name:=<unique-map-name>
```

6. Check the map saved in the following path:

```
cd ~/robot_settings/robot_data/maps/<unique-map-name>
```

### 7.1.2. 3D Map

1. Establish an SSH connection to the robot or open a terminal using a remote desktop via RPD.

2. Activate 3d mapping by setting `ROBOT_LOCALIZATION_MODE` in *robot_params*.

```
sed -i \
  -e 's/^export ROBOT_LOCALIZATION_MODE=.*/export
ROBOT_LOCALIZATION_MODE="mapping_3d"/' \
  "$HOME/robot_settings/robot_params/common/localization_params.env"
```

Note: ensure you set the map name before starting, cause on the following steps will be overwritten the name that is defined. Use the following command to set the `my_demo_map` name.

```
sed -i \
  -e 's/^export ROBOT_LOCALIZATION_MAP_NAME=.*/export
ROBOT_LOCALIZATION_MAP_NAME=<unique-map-name>/' \
  "$HOME/robot_settings/robot_params/common/localization_params.env"
```

3. Relaunch Robotnik's software to apply the changes:

```
~/bringup.sh
```

4. In the remote desktop, open RViz2 to verify that the robot is mapping. And start mapping the workaround areas.

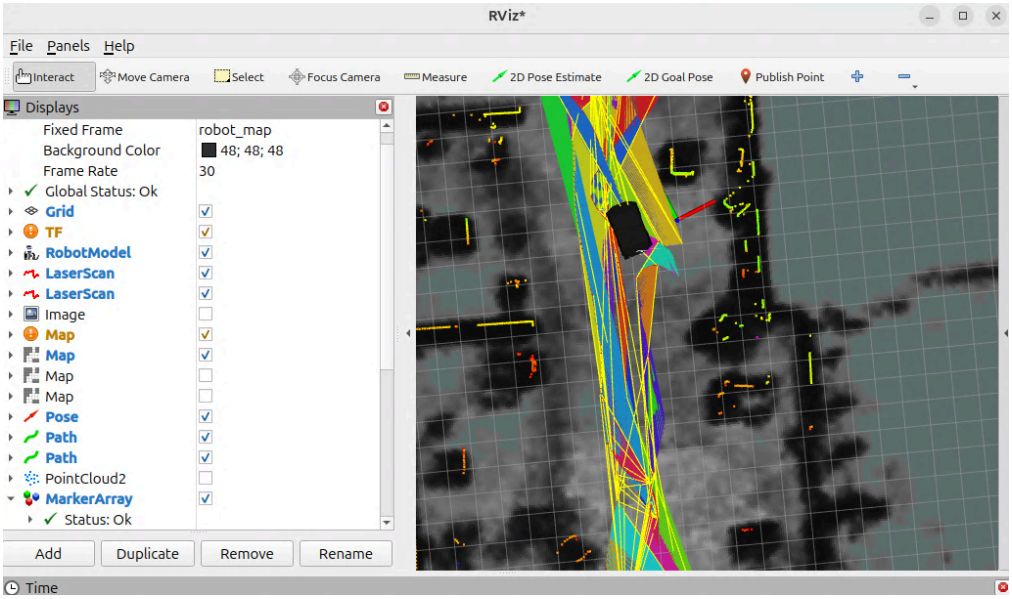


*Figure - Well created 3d map in rviz2*

5. To save the map and optimize the map, open screen localization and stop.

```
screen -S localization -X stuff $'\003' && screen -r localization
```

Wait for the “Export map process completed successfully” info message to be shown.

6. Check the map saved in the following path:

```
cd ~/robot_settings/robot_data/maps/<unique-map-name>
```

## 7.2. Load a previous map

1. Set the map created

```
sed -i \
  -e 's/^export ROBOT_LOCALIZATION_MAP_NAME=.*/export
ROBOT_LOCALIZATION_MAP_NAME=<unique-map-name>/' \
  "$HOME/robot_settings/robot_params/common/localization_params.env"
```

2. Set desired mapping mode by setting `ROBOT_LOCALIZATION_MODE` in *robot_params*.

    a. Option: 2D localization

```
sed -i \
  -e 's/^export ROBOT_LOCALIZATION_MODE=.*/export
ROBOT_LOCALIZATION_MODE="localization_2d"/' \
  "$HOME/robot_settings/robot_params/common/localization_params.env"
```

    b. Option: 3D localization

```
sed -i \
  -e 's/^export ROBOT_LOCALIZATION_MODE=.*/export
ROBOT_LOCALIZATION_MODE="localization_3d"/' \
  "$HOME/robot_settings/robot_params/common/localization_params.env"
```

3. Relaunch Robotnik's software to apply the changes:

```
~/bringup.sh
```

4. In the remote desktop, open RViz2 to verify that the robot is localized in the selected map. Send init_pose using the `2D Pose Estimate` tool to the "/robot/initialpose" topic.

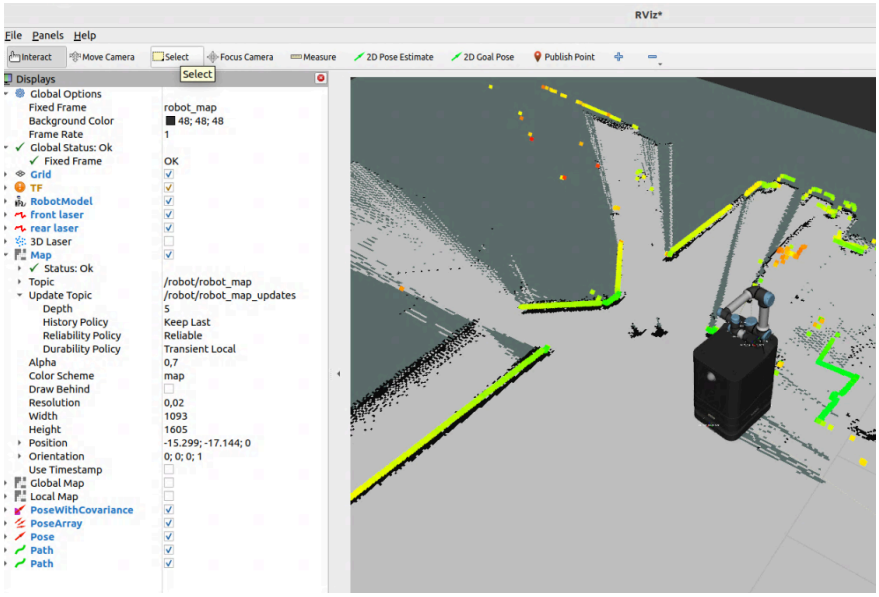


*Figure - Loaded 2d map in RViz2*

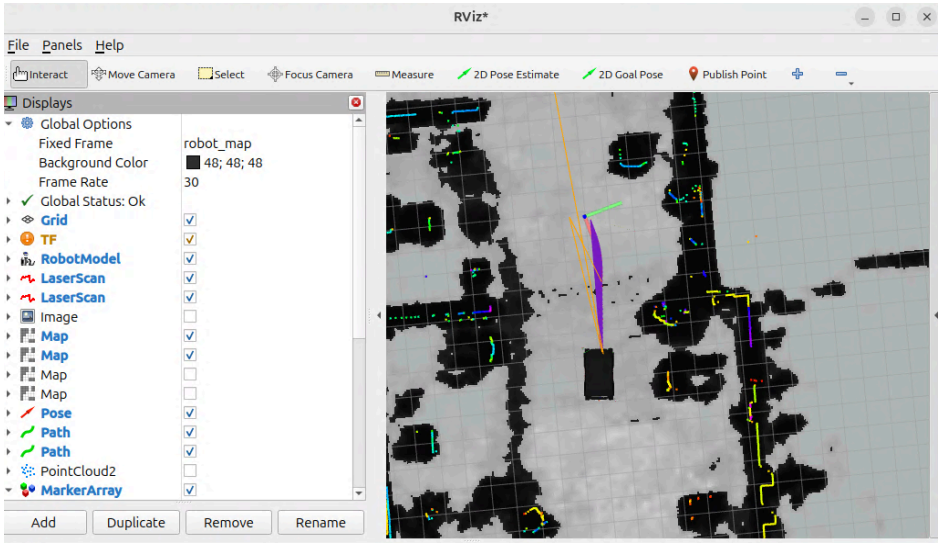


*Figure - Loaded 3d map in RViz2*

## 7.3. Edit a map

A map is used to localize the robot and compute paths to a target pose. Once the map is created, it can be edited to remove noise from mapping, especially in 3D maps.

All maps are located at: ROBOT_LOCALIZATION_MAPS_PATH with the following structure:

```
$ROBOT_LOCALIZATION_MAPS_PATH
├── my_2d_map
│   ├── map.pgm
│   └── map.yaml
└── my_3d_map
    ├── cartographer_3d.pbstream
    ├── map.pgm
    └── map.yaml
```

- map.pgm: raw pgm file:
  - 2D: map used to localize the robot and navigate.
  - 3D: projected 3d map, used to navigate.
- map.yaml: auxiliary file to configure the map with map_server.
- cartographer_3d.pbstream: 3d map to localize (only on 3d maps).

The ROS map supports only three values: occupied, free, and unknown. There are two main ways of editing and cleaning the map:

a. Configure thresholds in the map.yaml file (the configuration file of the map_server) to correctly classify pixels. This is especially important for 3D maps, which usually contain more noise than 2D maps.

b. Edit the image manually with a tool such as GIMP, adjust the pixel values, and save the file. White pixels represent free space, and black pixels represent occupied space (obstacles).

# 8. Navigation module

This module is responsible for autonomous and primitive motions by providing planning and execution. It is composed of two parts:

- **Autonomous Navigation:** Uses the map and estimated position provided by the localization module to calculate trajectories for autonomous motion.
- **Motion Primitives:** Performs atomic actions such as docking or one-axis movements.

This module provides 3 different services:

- **move:** This node allows one-axis movements using odometry.
- **dock:** This node allows docking to a specific frame using the perception module.
- **nav2:** The DWA local planner and Navfn as a global planner are being used.

## 8.1. Autonomous Navigation

1. In the remote desktop, open RViz2.

2. Make sure that the localization module has loaded the desired map. When the localization initiates, the robot cannot be localized automatically. You need to use *the 2D Pose Estimate tool* to set an initial position estimation. Also note that the fixed_frame configured in RViz2 is “robot_map”.

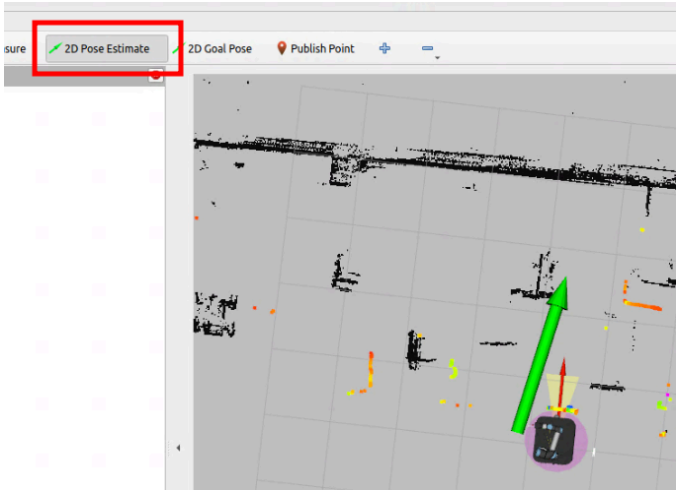


*Figure - Initpose in rviz2*

Initpose can also be set via ROS2 topic:

```
ros2 topic pub /robot/initialpose
geometry_msgs/msg/PoseWithCovarianceStamped <TAB + TAB> -1
```

Or via a ROS2 service:

```
ros2 service call /robot/set_initial_pose nav2_msgs/srv/SetInitialPose <TAB
+ TAB>
```

Remember to set *frame_id* as 'robot_map' when the msg is autocompleted when clicking <TAB + TAB>.

3. Use the navigation module to send autonomous goals to the robot using *2D Goal Pose*:

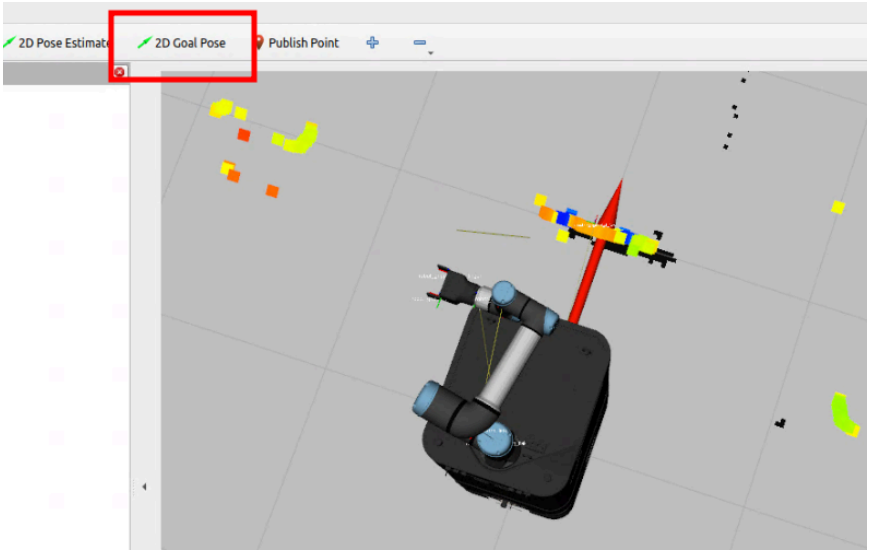


*Figure - Send goal pose in rviz2*

A goal can also be set via the ros2 topic:

```
ros2 topic pub /robot/goal_pose geometry_msgs/msg/PoseStamped <TAB + TAB>
```

Remember to set *frame_id* as 'robot_map' when the msg is autocompleted when clicking <TAB + TAB>.

4. Navigate through different positions on the map.

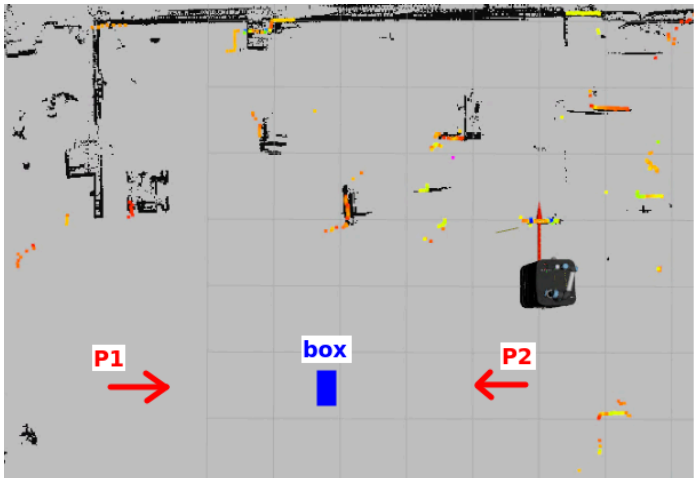


*Figure - Navigation sequence*

5. Use the following topic to get the robot's position on the map:

```
ros2 run tf2_ros tf2_echo robot_map robot_base_footprint | grep T -2
```

6. Use that information to save new points:

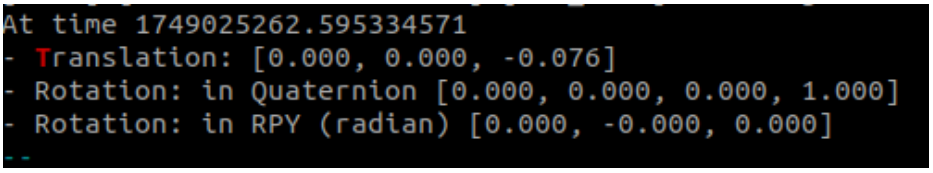


*Figure - Current robot position*

# 8.2. Motion Primitives

## 8.2.1. Move primitive

This feature allows the robot base to move a specified distance along a single axis using odometry.

```
ros2 action send_goal /robot/move robotnik_navigation_msgs/action/Move
"goal:
  x: 0.1
  y: 0.0
  theta: 0.0
maximum_velocity:
  linear:
    x: 0.0
    y: 0.0
    z: 0.0
  angular:
    x: 0.0
    y: 0.0
    z: 0.0"
```

## 8.2.2. Dock primitive

This feature allows the robot base to dock onto a specified frame using odometry.

- **Dock frame**: The target frame in the TF tree is used as the docking reference.
- **Robot dock frame**: The reference frame of the robot base, typically *robot_base_footprint*.
- **Dock offset**: The desired distance to maintain from the target docking frame.
- **Maximum velocity**: The speed limit enforced during the docking process.

```
ros2 action send_goal /robot/accurate_smooth_dock/dock
robotnik_navigation_msgs/action/Dock "dock_frame: 'robotnik_marker_1'
robot_dock_frame: 'robot_base_footprint'
dock_offset:
  x: -0.2
  y: 0.0
  theta: 0.0
```

```
maximum_velocity:
  linear:
      x: 0.4
      y: 0.0
      z: 0.0
  angular:
      x: 0.0
      y: 0.0
      z: 1.0"
```

# 9. Behavior module

The behavior module provides advanced functionalities for managing and coordinating interactions between multiple modules. These include:

- **Charge**: Moves the robot to the charging station, ensuring correct TF perception, docking alignment, battery charging, safety checks and recovery procedures.

- **Uncharge**: Moves the robot away from the charging station, preparing it to resume other tasks.

## 9.1. Charge

This feature allows the robot base to charge the robot at the charging station. It orchestrates different modules and nodes to achieve this purpose

- Perception: check that the charging station TF exists.
- Navigation: uses the dock and move primitive to move the robot into the charging station.
- Base: the charging information from the power system to ensure that the robot is charging.

The action enables setting different configurations. **Use this action and parameters** to send the robot to the **charging station**:

```
ros2 action send_goal /robot/charge robotnik_navigation_msgs/action/Charge
"dock_frame: 'robotnik_marker_1'
robot_dock_frame: 'robot_base_docking_contact'
dock_offset: 0.1
retries: 1" -f
```

### 9.1.1. Workflow process

The charging procedure begins by changing the mode of the **safety lasers**, if the robot is equipped with them. Next, the robot performs a **docking** action towards the marker of the docking station, stopping at an offset distance determined by the parameter *dock_offset*.

Once docking is complete, the robot **moves forward** to make contact with the charging connectors. Then, it **waits** for the battery **charge** to be detected**.**

### 9.1.2. Recovery process

If charging is not detected, the robot enters a recovery mode, which depends on the *retries* parameter. The robot will attempt the charging process as many times as specified by this parameter. In each retry, the robot will **move backward** a certain distance and then restart the procedure.

If all retries fail, the robot will wait longer on the final attempt to detect charging. If charging is still not detected, the robot will end the procedure in a backward position, indicating a failure. At this point, modifications to the configuration file may be necessary.

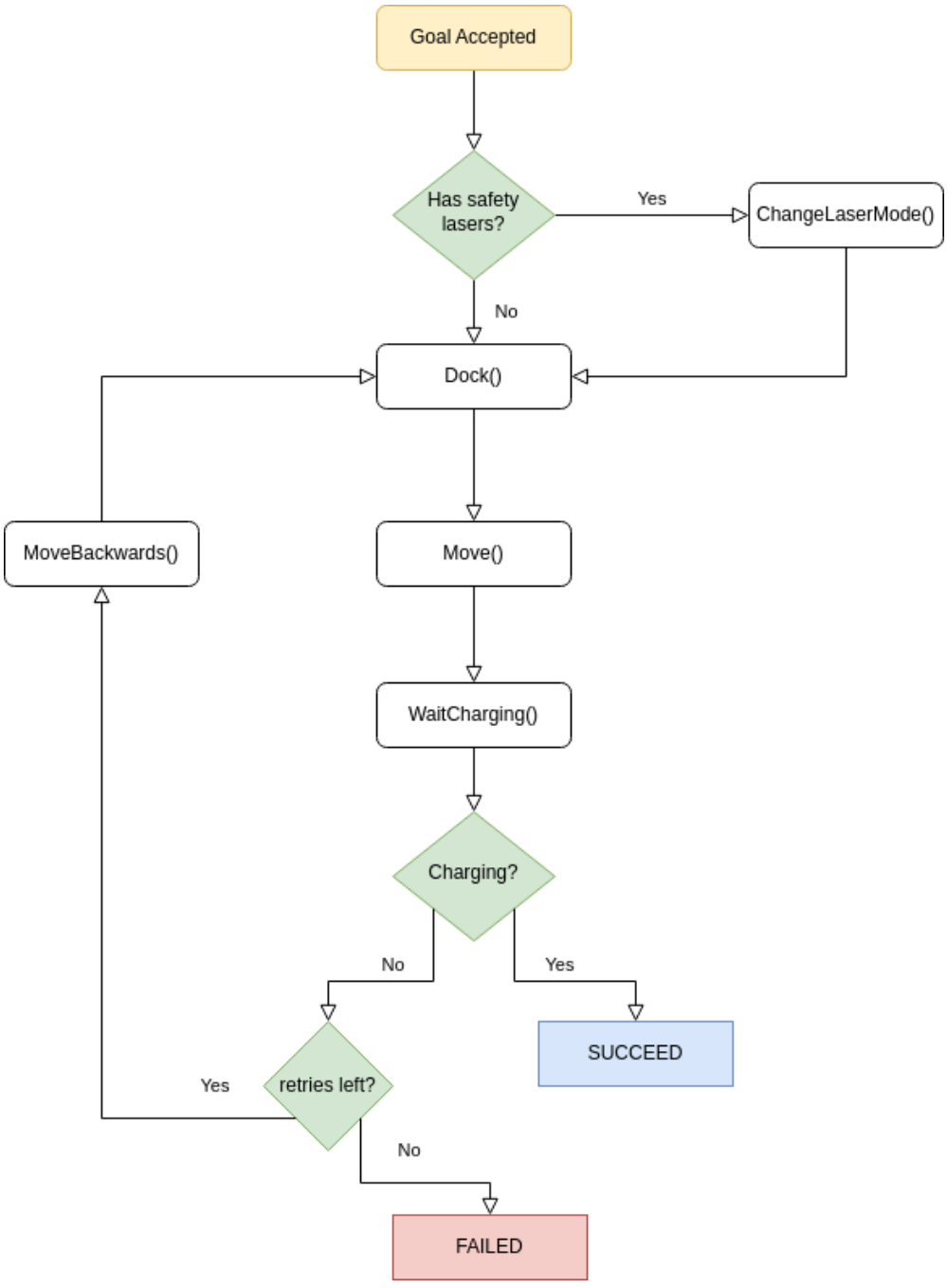


*Figure - Charge behavior diagram*

## 9.2. Uncharge

This feature allows the robot base to uncharge by moving away from the charging station.

```
ros2 action send_goal /robot/uncharge
robotnik_navigation_msgs/action/Uncharge -f
```

# 10. Manipulation module

The manipulation module is responsible for handling and interacting with physical objects. It also provides access to hardware components such as **arms, grippers** and other related elements, including **lifting platforms** to extend the reach and dedicated cameras for collision detection.

Please, take into account that the information included in this chapter will only apply to the devices integrated in your specific robot.

**The module is composed of three main functionalities:**

- **Arm** – Enables ROS 2-based control of the robotic arm by interfacing directly with the hardware.
- **Gripper** – Provides ROS 2 control of the end effector (gripper) through a custom hardware interface.
- **Lift** – Controls the lifting platform to adjust the vertical position of the arm, also through hardware access.

This module provides the following functionalities:

- **UR Driver** – Interfaces with Universal Robots arms, allowing ROS 2-based motion control via the hardware.
- **Robotnik Gripper Driver** – Provides control of the gripper using ROS 2 communication and URCap commands.
- **Lift Driver** – Manages lift platform positioning through direct hardware communication under ROS 2

## 10.1. Startup Verification

This section defines how to verify that the UR arm and gripper are properly launched and running and explains how to recover them in case of disconnection.

1. Ensure that the UR arm is initialized when the base boots. Check the rear panel to verify that the arm's LED is lit green. If the UR arm is not initialized, check for a dedicated startup button in the robot's control panel and press it if available. In other cases, please contact support.

2. Release all emergency buttons and rearm the robot.

3. Connect to Polyscope and check that the robot is in remote mode. This mode limits direct control through PolyScope but increases safety for remote interfaces such as ROS2.

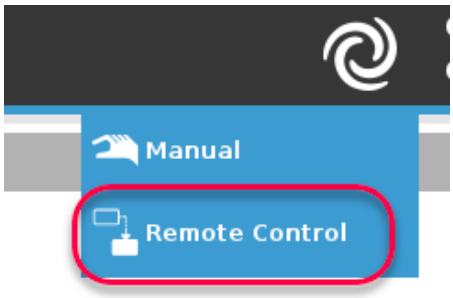


*Figure - Control modes in Polyscope*

In case of wanting to modify UR script or any arm parameter, or manually run programs, change to Automatic or Manual control.

4. Check if the arm is ready in the Polyscope using VNC. At the startup, the arm may initiate automatically.

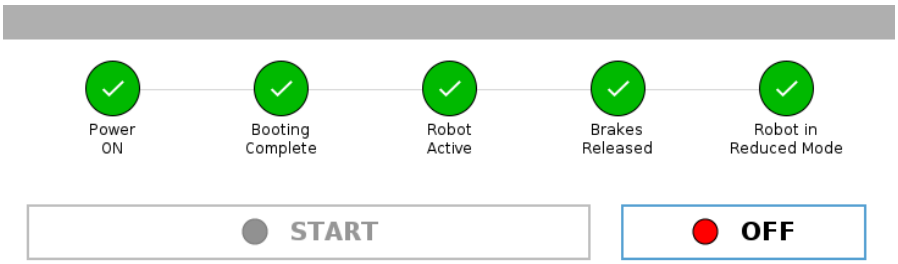


*Figure - UR arm startup ready*

In case of **manual startup** needed**,** press “start” / “on” twice following the Polyscope indications:

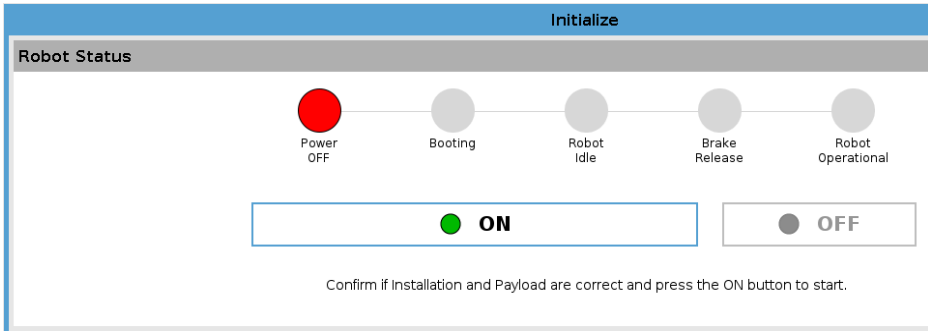


*Figure - UR arm startup power off*

5. Check that the *main* program is loaded and it's running. This program is used to control the Polyscope from the robot PC. At the startup, this program is launched automatically.

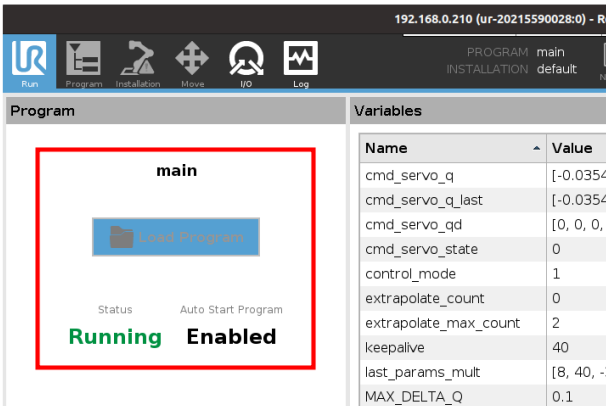


*Figure - Polyscope main program*

In case of **manual running,** press the Play button:

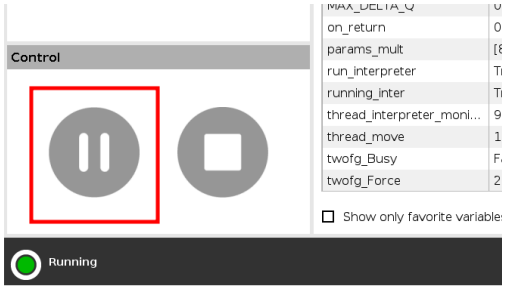


*Figure - Polyscope play program*

6. Check on the robot that the module is running properly.

```
screen -r manipulation
```

If the manipulation module is ready to control the UR arm and the gripper, the message "*Robot connected to reverse interface*" will be received.

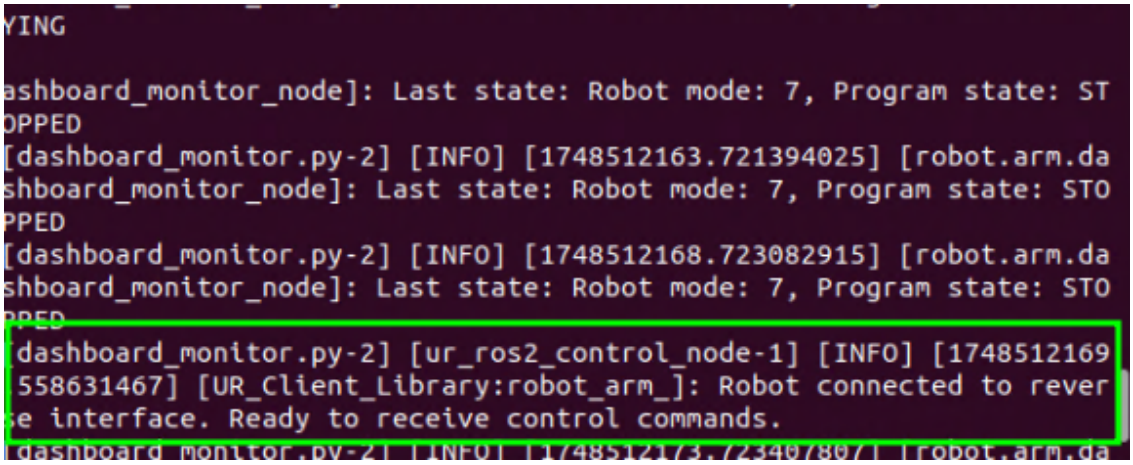


*Figure - Manipulation ready to receive commands*

# 10.2. Arm Control

## 10.2.1. Joint trajectory controller

This section describes how to move the arm using gui for joint_trajectory_controller:

1. Open a terminal on your computer.
2. Connect to the robot via RDP.
3. Once connected, open a terminal and execute:

```
ros2 run rqt_joint_trajectory_controller rqt_joint_trajectory_controller
--ros-args -r __ns:=/robot
```

## 10.2.2. MoveIt!

The arm can be controlled using MoveIt!. Since each robot typically has a specific URDF file, a custom MoveIt! configuration is used for collision checking. This configuration is identified by the robot's serial number. The corresponding package can be found here:

```
cd ~/robot_ws/src/<serial-number>_moveit/
```

**Note:** grippers are not included in the default MoveIt! control configuration.

1. On a remote desktop via RDP, open a new terminal and launch MoveIt!

```
docker compose up
```

2. Use the interactive marker to set new goals and plan and execute trajectories.

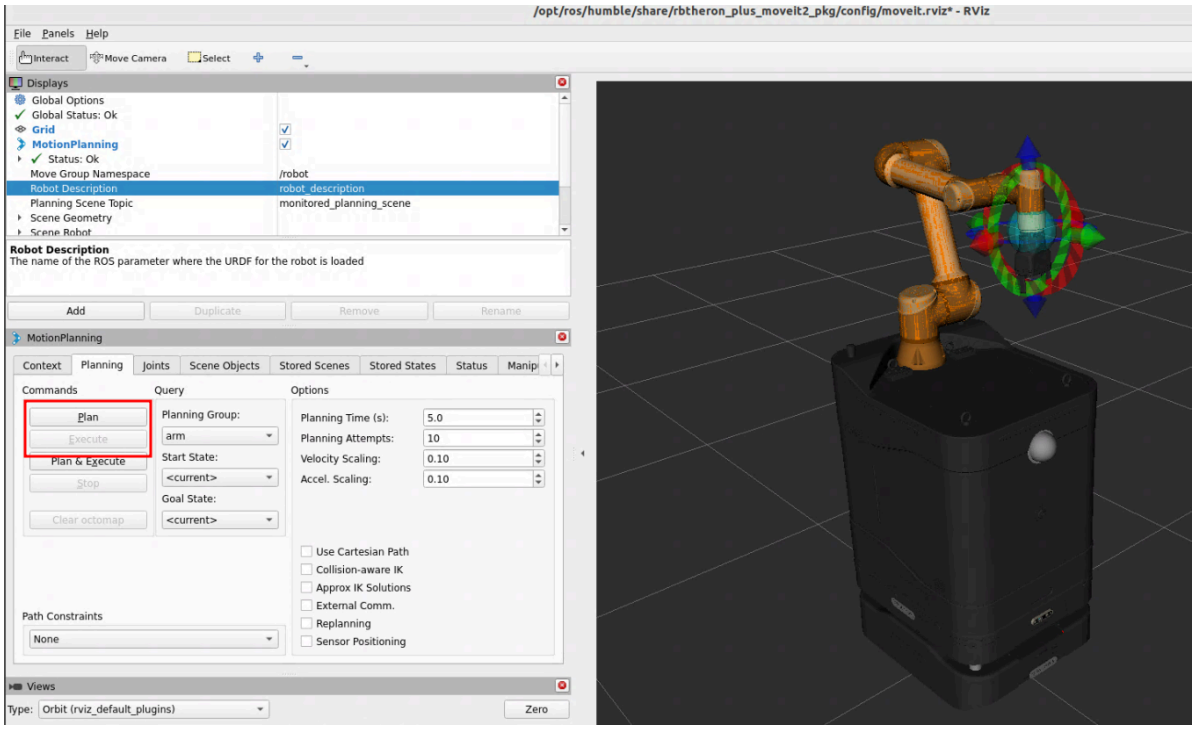


*Figure - Moveit!*

## 10.3. Gripper control

The gripper is controlled by a ROS 2 controller through a custom hardware interface that uses UR communication to send Polyscope commands to the gripper URCap, enabling actions such as grasping or releasing.

Use the following action to set a specific position value for the gripper:

```
ros2 action  send_goal  /robot/gripper/gripper_controller/gripper_cmd
control_msgs/action/ParallelGripperCommand "command:
  header:
      stamp:
      sec: 0
      nanosec: 0
      frame_id: ''
  name: []
  position: [0.035]
  velocity: []
```

## 10.4. Lift control

The lift device is controlled by a node which subscribes to setpoint topics and forwards the command through low level communication with the lift unit.

For example, to send movement (0.2 m) execute the following command:

```
ros2 topic pub /robot/lift/setpoint std_msgs/msg/Float64 "data: 0.2"
```

Current position is published as joint state (robot_lift_joint) and pos topic.

```
ros2 topic echo /robot/lift/pos
data: 0.19988069876438008
---
data: 0.19988069876438008
---
data: 0.19988069876438008
---
```

# 11. Robotnik ROS2 API

## 11.1. Topics

| Topic | Module | Description |
|---|---|---|
| /robot/robot_description | Base | Provides robot description URDF |
| /robot/robotnik_base_control/cmd_vel | Base | Set the velocity on the base before sending it to the kinematics controller |
| /robot/robotnik_base_control/odom | Base | Provides robot odometry |
| /robot/robotnik_io_controller/io | Base | Provides states of GPIO motor drivers |
| /robot/pad_teleop/cmd_vel_unstamped | Base | Provides velocity command sent by the pad controller |
| /robot/imu/data | Base | Provides filtered imu orientation |
| /robot/battery_estimator/data | Base | Provides battery level |
| /robot/charge_manager/status | Base | Provides charging/uncharging data |
| /robot/front_laser/scan | HW | Provides front laser scan if exists |
| /robot/rear_laser/scan | HW | Provides rear laser scan if exists |
| /robot/top_3d_laser/points | HW | Provides top laser pointcloud if exists |
| /robot/front_3d_laser/points | HW | Provides front laser pointcloud if exists |
| /robot/front_rgbd_camera/color/image_raw | HW | Provides front camera RGB image if exists |
| /robot/front_rgbd_camera/depth/image_rect_raw | HW | Provides front camera depth image if exists |
| /robot/robot_map | LOC | Provides the map |
| /robot/amcl_pose | LOC | Provides robot pose estimation in the map |

| | | |
|---|---|---|
| /robot/initialpose | LOC | Set initial localization for AMCL |

## 11.2. Services

| Service | Module | Description |
|---|---|---|
| /robot/robotnik_base_control/set_odometry | Base | Set odometry, used to reset it |
| /robot/robotnik_io_controller/set_digital_output | Base | Set digital output through motor drivers |